CÉGEP De Chicoutimi

LIVRABLE 2 : CAHIER DE CONCEPTION

PARTAGE DE DÉPENSES - Dév. App. Avancé | 420-D60-CH | Gr. 01

Henri Pitre & Tommy-Frédérique Bernaquez
2026-02-23

TABLE DES MATIÈRES

TABLE DES MATIÈRES 2
1. PERSPECTIVE GLOBALE 4
1.1 Rappel de fonction 4
1.2 Contraintes 4
1.3 Définition de l'architecture de l'application 4
2. MODELISATION ORIENTEE OBJET 5
2.1 Encapsulation 5
2.2 Collections et listes d'objets 5
2.3 Héritage 5
2.4 Rôles par classe 5
2.5 Classes MODEL 5
2.6 Diagramme de classes 6
2.7 Validation et appel de contraintes importantes 7
3. GESTION DES DROITS 8
3.1 Explication des permissions 8
3.2 Tableau sommaire 8
4. STRATÉGIE MATHÉMATIQUE 9
4.1 Formule mathématique 9
4.2 Pseudo-code 9
5. EXIGENCES DE CONCEPTION 9
6. CONCEPTION DE L'INTERFACE 10
6.1 Principes de conception 10
6.2 Architecture de l'interface 10
6.3 Maquettes fonctionnelles 11
6.3.1 Interface de création d'activité 11
6.3.2 Écran de vue d'ensemble personnel 12
6.3.3 Création d'une nouvelle dépense liée à une activité 13
6.3.4 Écran de règlement de dette 14
6.3.5 Écran de consultation d'activité 15
6.4 Responsivité et redimensionnement 15
7. Stratégies de test 16

7.1 Philosophie de test 16
7.2 Tests unitaires 16
7.2.1 Objectifs 16
7.2.2 Éléments testés 16
7.3 Tests d'intégration 16
7.4 Tests fonctionnels 16
7.5 Tests d'acceptation 16
8. Analyse des risques 17
8.1 Incohérence Données XML 17
8.2 Grand nombre de données 17
8.3 Mauvaise gestion des droits 17
9. APPROBATIONS 17

1. PERSPECTIVE GLOBALE

1.1 Rappel de fonction

L'application permet la gestion d'activités regroupant plusieurs participants et plusieurs dépenses, avec calcul automatique des soldes et minimisation algorithmique des transferts financiers.
Objectif principal : réduire le nombre total de transactions nécessaires au règlement complet.

1.2 Contraintes

- Support 1 à 1000+ utilisateurs.
- Enregistrement local en XML.
- L'application doit être assez optimisée et efficace pour un nombre d'utilisateurs conséquent

1.3 Définition de l'architecture de l'application

Nous utiliserons l'architecture ***MVVM***, afin d'assurer une séparation claire :
- Model : Instancier/appliquer la logique métier et définition d'objets
- ViewModel : Liaison et validation des données pour ensuite joindre le Model au View
- View : Interface utilisateur

2. MODELISATION ORIENTEE OBJET

2.1 Encapsulation

Les propriétés critiques (soldes, état IsClosed, isActive) sont protégées par des méthodes métier empêchant toute modification directe incohérente.

2.2 Collections et listes d'objets

Certaines listes auront une accessibilité en GET presqu'entièrement globale puisque celles-ci sont nécessaires au fonctionnement de l'application. Par exemple, la liste d'activités créées et la liste de participants devra être partagée dans la plupart des différentes sections pour afficher, modifier ou encore instancier les détails d'une personne, d'un solde ou autre.

- Activite contient une collection de Depense.
- Depense contient une collection de ParticipantsPayeurs.
- Reglement est associé à une Depense.

2.3 Héritage

Par le principe de base de POO, l'héritage est priorisé de sorte que tout objet existant peut avoir un objet parent pour partager les propriétés entre objets différents. De même façon, tous les ViewModel héritent de BaseViewModel, qui à son tour implémente INotifyPropertyChanged.

2.4 Rôles par classe

Participant : gestion identité, historique.
Activite : gestion état, agrégation des dépenses.
Depense : gestion montant + validation logique.
Reglement : traçabilité des remboursements.
XMLService : persistance unique responsabilité (Handler du fichier XML).
RapportService : logique mathématique de solde et de comptes rendus

2.5 Classes MODEL

Participant

- username,
- pwd,
- ActivitiesOwned{get},
- DepensesOwner{get}

Activite:

- Owner,
- Desc,
- ParticipantsConcernes,
- DateTime.Now,
- Depenses{get}
- (toute get;set;)
- IsClosed : Si fermé, peux pu rien ajouter/modifier
  Une dépense peut être réglée même si une activité propriétaire est Closed

Depense:

- Owner,
- Desc,
- ParticipantsPayeurs(FromParticipantsConcernes),
- TimeCreated(Now),
- ReglementAssocié nullable,
- isActive {get return ? ReglementAssocie.isNull}
  Une dépense peut être réglée même si une activité propriétaire est Closed

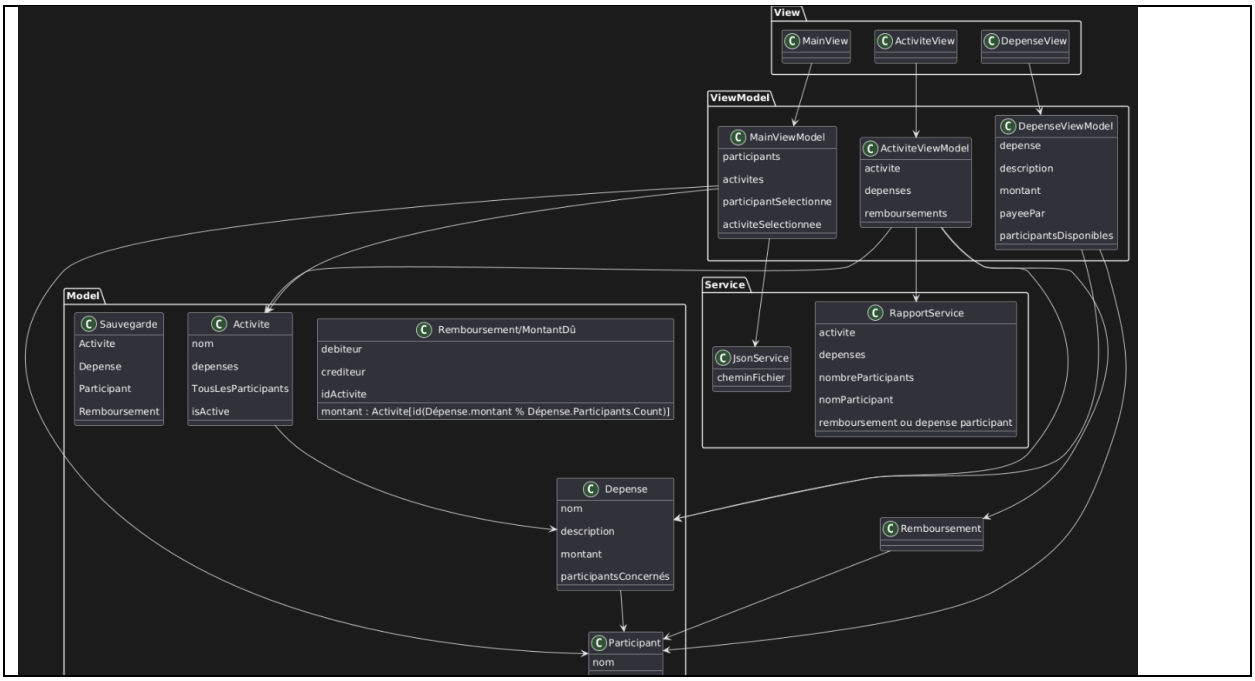
LoggedObject:

- List<Activites.Where isClosed ==true> ***Une dépense peut être réglée même si une activité est Closed***
- List<Depenses.Where isActive == false>

Reglement:

- Owner,
- DepenseAssociee,
- DateTime.Now

2.6 Diagramme de classes



2.7 Validation et appel de contraintes importantes

- Interdiction de modifier une activité si IsClosed = true.
- Une dépense réglée devient isActive = false.
- À la fin des soldes, le montant doit absolument être >= 0.
- Interdiction suppression sans droits appropriés.

3. GESTION DES DROITS

3.1 Explication des permissions

Chaque participant est modérateur propriétaire de ce qu'il crée. Ainsi, n'importe qui peut créer une activité s'il a accès au programme contenu sur le fichier local. Celui-ci devra se connecter avant tout afin d'identifier le créateur de l'activité. Ensuite, celui-ci seul aura la permission de modifier et ajouter des participants à l'activité. Finalement, chaque participant inscrit à une activité aura la permission de créer des dépenses, et sera admin de celle-ci. L'admin de l'activité sera également admin de la dépense.

3.2 Tableau sommaire

Action	Participant	Admin Activité	Admin Dépense
Créer activité	Oui	-	-
Supprimer/Fermer activité	Non	Oui	Non
Modifier dépense	Non	Oui	Oui
Régler dépense	Oui	Oui	Oui

4. STRATÉGIE MATHÉMATIQUE

4.1 Formule mathématique

Une dépense peut être crée en ayant des participants ayant déjà payé une partie. Ainsi, un dîner de 10 personnes coûtant 80$ pourrait avoir été payé par deux personnes à 40$ chaque, alors que 8 personnes sur ces 10 souhaitent contribuer équitablement.

$$Part\ individuelle = \frac{80(Montant\ Payé\ Total)}{8\ (Nb\ Contributeurs)} = 10\$$$

Pour 2 payeurs qui veulent équilibrer la part de tous :

$$Part\ des\ contributeurs\ = \frac{30(SurplusPayeurs)}{3\ (Contributeurs\ Restants)} = 10\ \$\ par\ contribueur$$

Chaque contributeur doit 10$. Cependant, deux personnes ont mis 40$ auparavant. Ainsi, 3 personnes n'ayant pas payé doivent 10$ à Payeur1, et de même pour Payeur2.

4.2 Pseudo-code

```
ListeCrediteurs ← participants où Solde > 0
ListeDebiteurs ← participants où Solde < 0

Trier ListeCrediteurs par Solde décroissant
Trier ListeDebiteurs par Solde croissant

C = Prochain Créditeur dans la liste

Pour chaque débiteur D:
    Tant que Solde(D) < 0:
        montant = min(|Solde(D)|, Solde(C))
        créer Remboursement(D vers C, montant)
        Solde(D) += montant
        Solde(C) -= montant
```

5. EXIGENCES DE CONCEPTION

Exigence	***Élément à concevoir***
Minimiser transferts	Algorithme mathématique
Gestion droits	Table gestion droits + validation ViewModel
Activité fermée	Validation IsClosed dans Model, et une gestion appropriée qui s'en suit

6. CONCEPTION DE L'INTERFACE

6.1 Principes de conception

L'interface sera conçue selon les principes de clarté, cohérence et efficacité. Chaque fonctionnalité sera accessible en maximum 3 clics. Les actions fréquentes (ajout de dépense, consultation de rapport) bénéficieront de raccourcis visuels. L'utilisation de codes couleur cohérents facilitera la distinction entre les différentes sections.

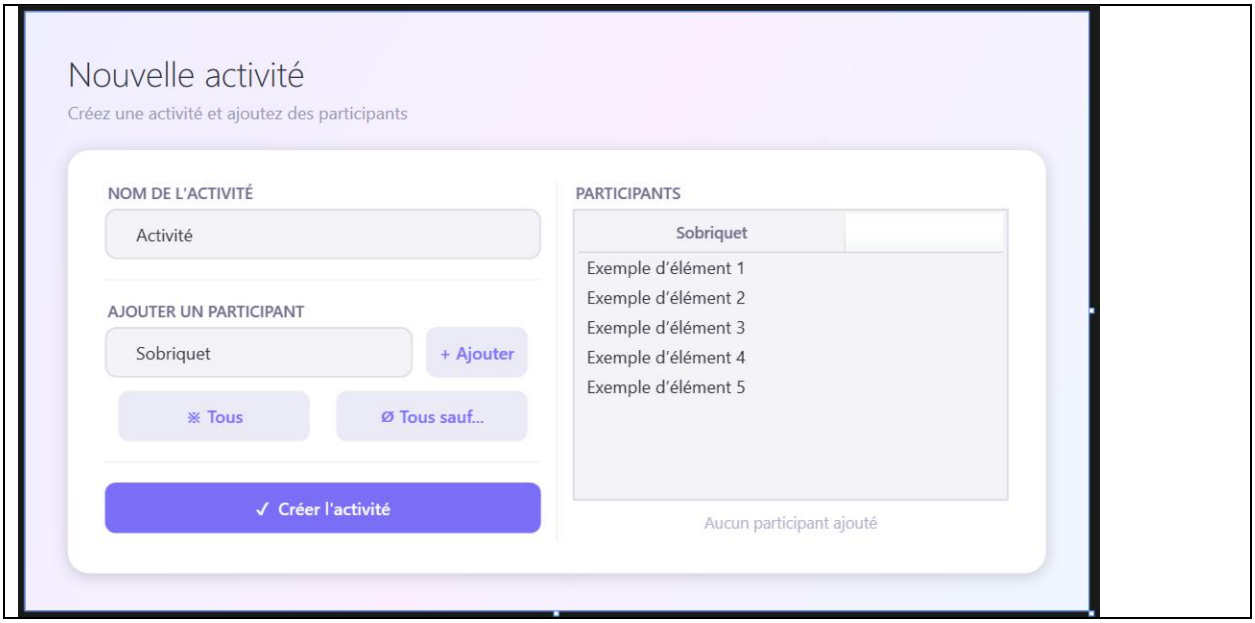
6.2 Architecture de l'interface

La fenêtre principale sera divisée en plusieurs zones fonctionnelles :

- Zone de contenu (centre) : Affichage principal des données et formulaires
- Zone de détails (droite) : Informations détaillées de l'élément sélectionné à la gauche
- Barre d'outils (haut) : Actions rapides
  (Onglets de différentes pages, Nouveau/Enregistrer/Modifer…)
- Barre d'état (bas) : Messages informatifs, état de l'application, validation

6.3 Maquettes fonctionnelles

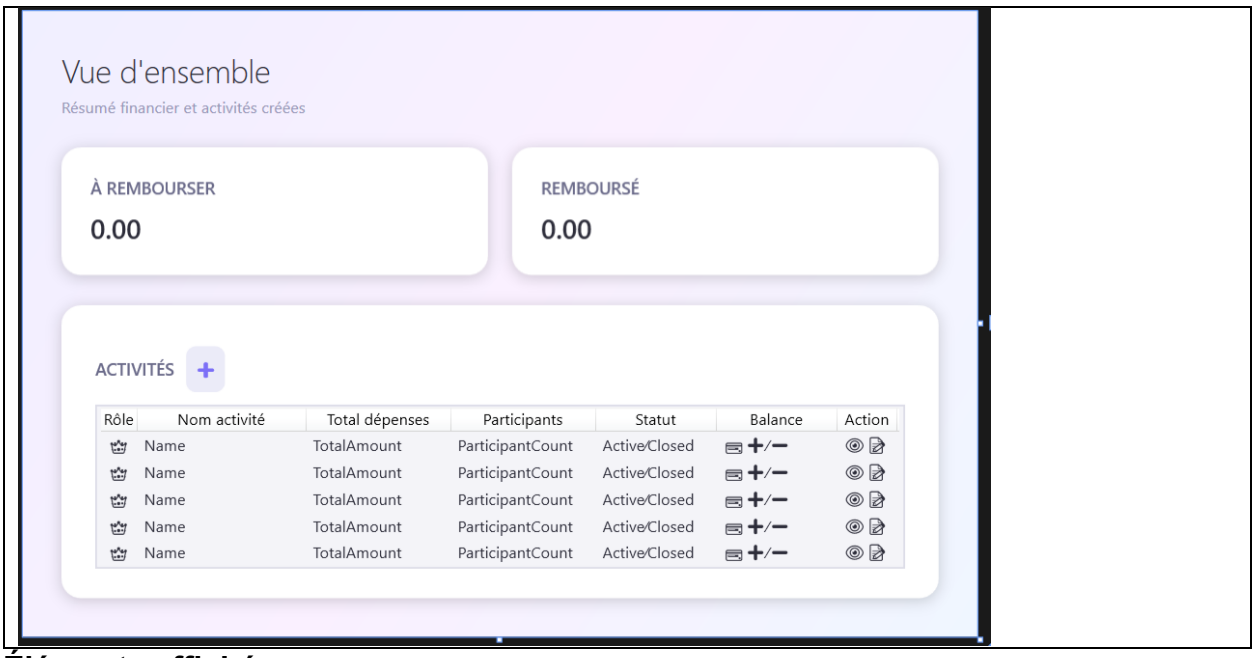
6.3.1 Interface de création d'activité



Éléments affichés :

- Liste des participants sélectionnés pour faire partie de l'activité
- Aire de texte pour nommer l'activité. Défaut : Activité_1, Activite_2…
- Boutons :
  - Ajouter un participant : En cliquant sur la barre de texte, une liste déroulante apparaît afin de sélectionner le participant à ajouter.
  - Tous : Permet d'ajouter tous les participants d'un coup à la liste.
  - Tous sauf… : Permet d'ajouter tous les participants sauf certains.
    - Selon la complexité, une interface de sélection de type InputBox s'ouvrira pour sélectionner les participants.
    - Alternativement, les participants déjà dans la liste seront exclus de cette même liste lorsque le bouton Tous sauf… est appuyé.
- Créer l'activité : Confirmation de la création

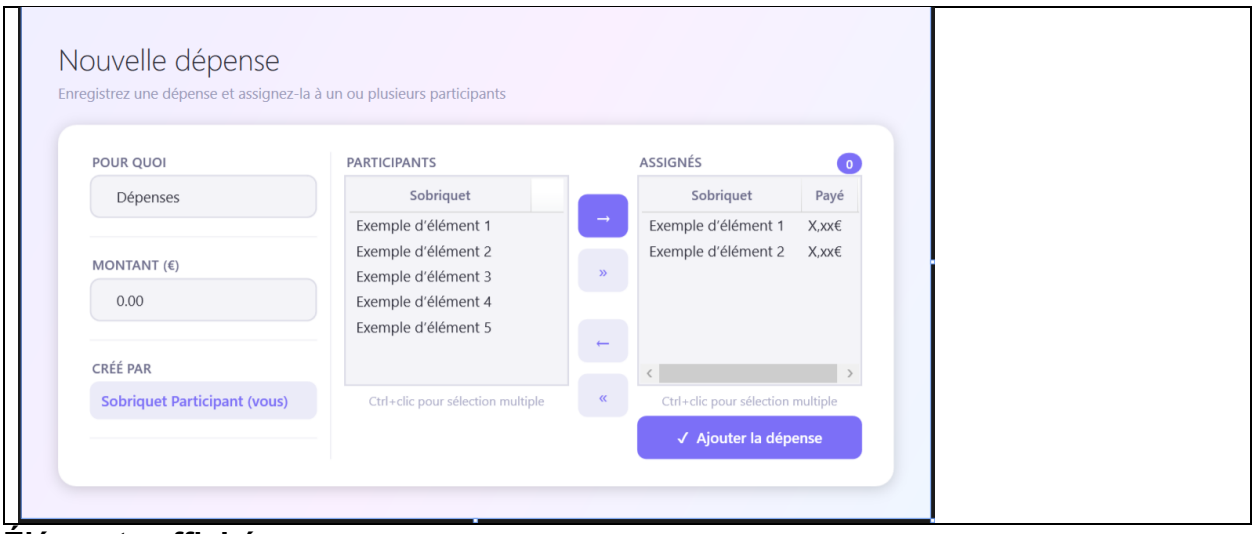
6.3.2 Écran de vue d'ensemble personnel



Éléments affichés :

- Liste des activités concernant l'utilisateur connecté
- Totaux personnels : Montant donnés et à donner
- Boutons : Ajouter Activité qui redirige vers Nouvelle activité
- Panneau de détails affichant : nom, dépenses, participants, ouverture, solde personnel
- Bouton pour modifier l'activité OU la voir si l'utilisateur n'est pas propriétaire

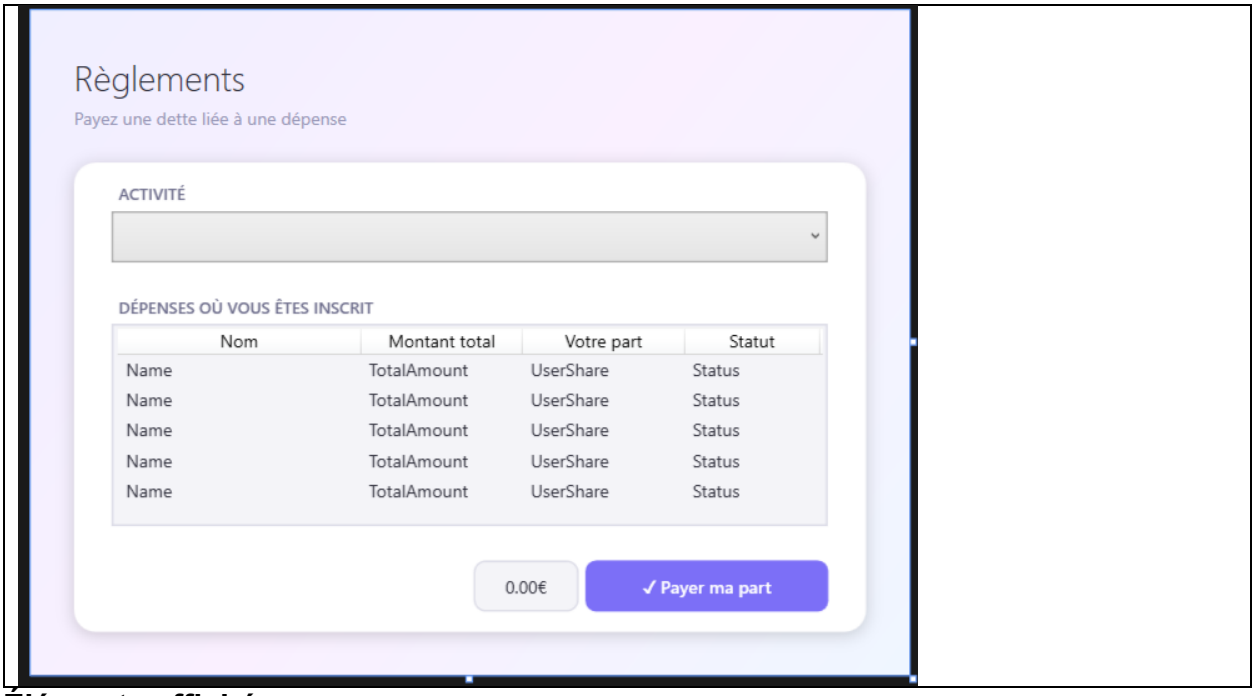
6.3.3 Création d'une nouvelle dépense liée à une activité



Éléments affichés :

- POUR QUOI : Titre / Description de la dépense
- Saisie du montant : Montant de la dépense totale
- Liste Gauche : Liste des participants existant dans l'activité parente
- Liste Droite : Liste des participants assignés à cette nouvelle dépense
  - Payé : Possibilité d'entrer le montant déjà payé par cette personne
    - Le programme s'occupera de balancer sa part dans l'équation et de l'égaliser.
- Bouton de confirmation Ajouter la dépense

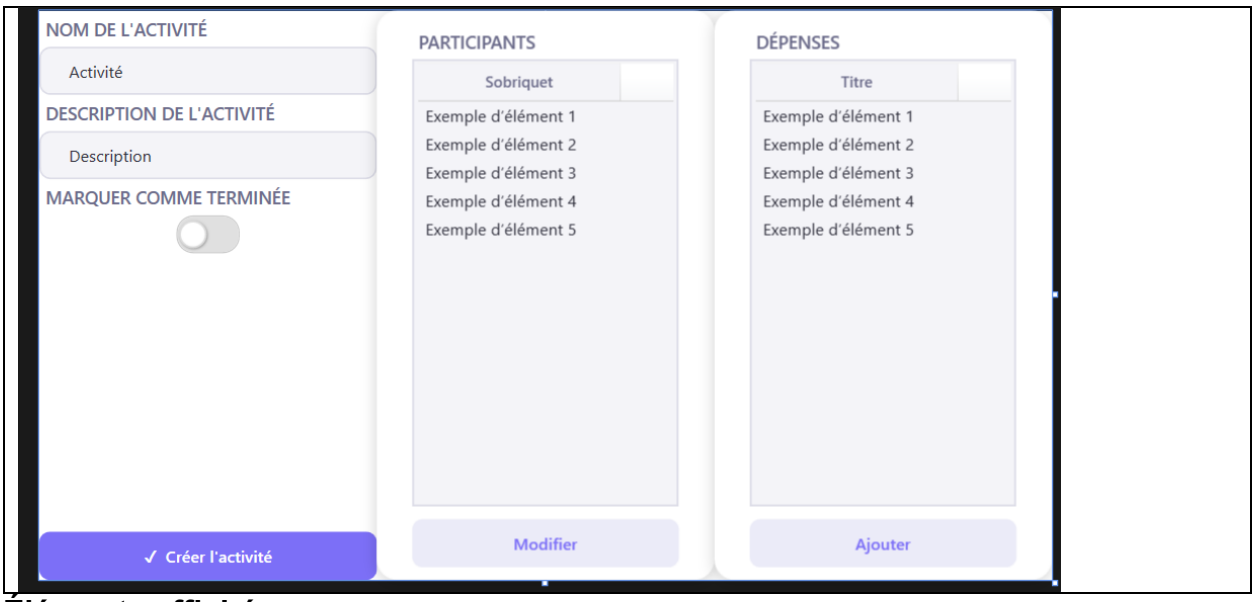
6.3.4 Écran de règlement de dette



Éléments affichés :

- ❖ Sélection de l'activité pour laquelle on veut payer
- ❖ Liste des dépenses nous concernant associée à l'activité sélectionnée
- ❖ Possiblité de payer une dépense :
    - Séléction dans la liste pour payer en plein
    - Saisie d'un montant pour payer en partie les dépenses
- ❖ Formulaire d'ajout/modification avec validation en temps réel

6.3.5 Écran de consultation d'activité



Éléments affichés :

- Modification du nom de l'activité
- Modification de la description de l'activité
- Possiblité de modifier la liste de participants
- Possibilité d'ajouter une dépense à l'activité
  - Si l'utilisateur est admin, possibilité de supprimer la dépense.

6.4 Responsivité et redimensionnement

L'interface sera conçue pour s'adapter dynamiquement au redimensionnement de la fenêtre. Les panneaux utiliseront des Grid et StackPanel WPF avec propriétés Width="*" et Height="*". En mode fenêtre réduite, les panneaux secondaires pourront être masqués ou transformés en onglets. Une taille minimale de fenêtre sera définie (1024x768) pour garantir la lisibilité.

7. Stratégies de test

7.1 Philosophie de test

Une stratégie de test complète et rigoureuse sera appliquée pour garantir la qualité du logiciel. Les tests couvriront tous les niveaux : unitaire, intégration, fonctionnel et acceptation. L'objectif est d'atteindre au minimum 80% de couverture de code pour les couches métier et données.

7.2 Tests unitaires

7.2.1 Objectifs

Valider le comportement individuel de chaque méthode et classe en isolation. Assurer la robustesse des validations, calculs et logiques métier.

7.2.2 Éléments testés

- Classe Participant : Validation du nom, unicité de l'Id
- Classe Activite : Ajout/suppression de dépenses, calcul du total
- Classe Depense : Validation du montant (> 0), description obligatoire
- CalculateurRemboursementSimple : Algorithme de calcul avec différents scénarios
- XmlDataManager : Sérialisation/désérialisation correcte des données

7.3 Tests d'intégration

Valider l'interaction correcte entre les différents composants du système. S'assurer que les données fonctionnent de bout en bout.

7.4 Tests fonctionnels

Valider que l'application répond aux exigences fonctionnelles définies dans le cahier des charges. Tester depuis l'interface utilisateur les fonctionnalités complètes.

7.5 Tests d'acceptation

Ces tests seront effectués en présence du client :

- Scénario complet soirée cinéma : Reproduire l'exemple du cahier des charges
- Test de capacité : Créer 10 participants, 5 activités, 25 dépenses
- Test d'ergonomie : Utilisateur novice doit accomplir toutes les tâches sans assistance
- Test de redimensionnement : Vérifier la lisibilité à différentes résolutions
- Test de précision : Vérifier calculs au cent près

8. Analyse des risques

8.1 Incohérence Données XML

Un problème pourrait survenir si la sérialisation ou désérialisation des données fonctionne incorrectement. L'intégralité d'un fichier doit donc être vérifiée avant de l'utiliser pour éviter ce problème. Il est donc nécessaire d'avoir un gestionnaire d'erreurs.

8.2 Grand nombre de données

Ce problème pourrait survenir si une liste occupe un espace en mémoire trop grande, ce qui engendrerait un ralentissement du système et des erreurs possibles, voire un bris de fonction dans l'application (crash).
Pour éviter ce problème, une limite de participants devrait être mise (dans les milliers), et l'historique des dépenses et activités devrait cesser d'être chargée si celle-ci contient trop de données. Alternativement, nous pourrions effacer les activités si celles-ci sont fermées et n'ont pas été consultées depuis un certain temps.

8.3 Mauvaise gestion des droits

Une mauvaise gestion de droits pourrait créer des scénarios où une dépense/activité se voit supprimée, entraînant ainsi une impossibilité de l'application à servir son utilité. Les permissions doivent donc être uniques et strictes, empêchant n'importe qui de faire n'importe quoi.

9. APPROBATIONS

Client	**Chef de projet**
Signature : _____ Date : _____	Signature : _____ Signature : _____ Date : _____